ORACLE

Autonomous Prediction Market Intelligence
& Execution Engine

Complete Engineering Specification & System Architecture Blueprint

Document Property	Specification
Project	ORACLE
Document version	1.0
Date	01 November 2026
Status	Proposed engineering baseline
Primary exchange	Polymarket
Research population	Up to 10,000 candidate wallets
Initial historical window	Rolling 90 days
Operating model	Autonomous intelligence with deterministic financial controls
Initial execution mode	Research, replay and paper trading
Intended deployment	Modular cloud infrastructure
Classification	Engineering design document

This document defines the target architecture and implementation requirements. It does not represent a completed software deployment, verified trading performance, guaranteed arbitrage or legal authorisation.

TABLE OF CONTENTS

1. Executive Summary
2. System Architecture
3. Technology Stack
4. Polymarket Data Ingestion Engine
5. Wallet Intelligence Engine
6. News and Event Intelligence Engine
7. Probability Intelligence Engine
8. Mispricing and Opportunity Detection
9. Cross-Outcome Arbitrage Engine
10. Kelly Capital Allocation Engine
11. Autonomous Agent Architecture
12. Execution Engine
13. Database Architecture
14. API and Service Interfaces
15. Security and Governance
16. Monitoring and Observability
17. Testing and Validation
18. Deployment Architecture
19. Development Roadmap
20. Repository Structure
21. Acceptance Criteria
22. Non-Negotiable Engineering Invariants
23. Future Development
24. Final Engineering Directive

1. EXECUTIVE SUMMARY

ORACLE is an autonomous prediction-market intelligence, research and execution platform designed to identify persistent informational and statistical advantages within prediction markets.

The system will analyse historical trading activity across up to 10,000 Polymarket wallets, identify statistically significant trading performance, reconstruct behavioural patterns, monitor live information, estimate independent event probabilities, detect market inefficiencies and calculate optimal risk-adjusted capital allocations.

ORACLE is not intended to function as a conventional rule-based trading bot. It is designed as a persistent autonomous agent capable of observing live market conditions, discovering and analysing successful market participants, identifying relevant developments in the external information environment, understanding relationships between news, events and prediction markets, developing independent probability estimates, identifying mispriced contracts and arbitrage opportunities, calculating position sizes using fractional Kelly optimisation, executing approved trading strategies, monitoring existing positions and changing conditions, and learning from historical outcomes and its own prediction errors.

The architecture separates intelligent reasoning from deterministic financial execution. An AI reasoning layer may interpret information, formulate hypotheses and coordinate research. It cannot independently bypass risk controls, modify capital limits, access signing credentials or submit unauthorised orders.

1.1 Core Objective

Discover repeatable market advantages and convert them into independently validated, risk-controlled trading strategies.

1.2 Primary Operating Principle

ORACLE does not copy successful traders. It discovers, validates and independently exploits measurable predictive advantages. Wallet activity is treated as an information signal rather than an unquestionable trading instruction.

1.3 System Objectives

Objective	Target
Wallet discovery	Up to 10,000 unique wallets
Historical analysis	Rolling 90-day research window
Performance reconstruction	Realised and unrealised P&L where reconstructable
Statistical analysis	Bayesian estimation, bootstrap and out-of-sample validation
Information intelligence	Continuous news and event monitoring
Market intelligence	Continuous market and order-book monitoring
Probability modelling	Independent probability estimation
Arbitrage detection	Multi-outcome and related-market analysis
Position sizing	Fractional Kelly and portfolio optimisation
Execution	Paper trading followed by controlled live execution
Learning	Continuous forecast and strategy evaluation

2. SYSTEM ARCHITECTURE

2.1 Architectural Philosophy

ORACLE must be modular, independently testable, horizontally scalable and capable of operating without requiring every subsystem to be online simultaneously. Each subsystem has a defined responsibility and interface. The platform uses event-driven communication between services, persistent storage for historical intelligence and deterministic APIs for financial calculations.

2.2 High-Level Architecture

External data sources (Polymarket Data API, CLOB, Market WebSockets, News/Official Sources, Wallet Activity) feed into a Data Ingestion Layer, then Normalisation and Validation. Data is persisted to PostgreSQL, Object Storage and an Event Queue. Intelligence workers (Wallet, News/Event, Market) feed Quantitative Intelligence, which drives the Probability Engine and Opportunity Detection. The Portfolio and Risk Engine feeds the Autonomous Agent, which passes through Policy Validation to the Execution Gateway (Paper Trading or Live Execution). Performance and Learning feedback loops close the system.

2.3 Architectural Layers

Layer 1: Data Acquisition — Collect market, wallet, order-book, news and event information.

Layer 2: Data Processing — Normalise, validate, deduplicate and persist incoming information.

Layer 3: Intelligence — Analyse wallet performance, event relationships, news and market behaviour.

Layer 4: Quantitative Analysis — Calculate independent probabilities, expected value, mispricing and arbitrage.

Layer 5: Risk Management — Determine position sizing, portfolio exposure and execution eligibility.

Layer 6: Autonomous Orchestration — Coordinate research, evaluate opportunities and manage persistent trading tasks.

Layer 7: Execution — Submit, monitor, modify and reconcile approved orders.

Layer 8: Learning — Measure performance and improve model quality through prospective evidence.

3. TECHNOLOGY STACK

The proposed initial technology stack is subject to benchmarking and operational requirements.

Component	Technology
Core backend	Python 3.12+
API framework	FastAPI
Frontend	Next.js
Frontend language	TypeScript
Database	PostgreSQL
Time-series storage	TimescaleDB or partitioned PostgreSQL
Cache	Valkey / Redis
Event streaming	Redis Streams or equivalent
Analytical processing	Polars, NumPy, Pandas
Statistical modelling	SciPy, scikit-learn, PyMC
Historical analytics	DuckDB
Large-scale storage	Parquet
Object storage	S3-compatible storage
Agent orchestration	Python-based persistent agent framework
Model integration	Configurable LLM and specialist forecasting models
Monitoring	Prometheus
Dashboards	Grafana
Tracing	OpenTelemetry
Deployment	Docker
CI/CD	GitHub Actions

4. POLYMARKET DATA INGESTION ENGINE

4.1 Objective

Build a reliable historical and real-time data foundation. The system must support discovery, historical reconstruction and continuous monitoring.

4.2 Data Domains

Market Metadata — Market ID, Condition ID, question, description, resolution criteria, outcome labels, token IDs, opening/closing/resolution times, status, category.

Wallet Trading Activity — Wallet address, transaction reference, market/token ID, direction, side, execution price, quantity, notional, timestamp, fees, type.

Position Information — Wallet, market, token, quantity held, average entry, current mark, realised/unrealised P&L, last update.

Order-Book Information — Token ID, bid/ask prices and quantities, timestamp, depth, spread, source sequence.

Historical Price Information — Market/token ID, price, timestamp, source, price type.

4.3 Ingestion Architecture

API Connectors → Request Scheduler → Rate Limit Controller → Data Retrieval → Schema Validation → (Valid? → Normalisation → Deduplication → Raw Archive + Canonical Database → Event Bus → Analytics Workers) or Dead Letter Queue.

4.4 Engineering Requirements

- Incremental data retrieval and cursor-based pagination
- Retry mechanisms with exponential backoff
- Duplicate detection and timestamp normalisation to UTC
- Schema versioning and raw payload preservation
- Data completeness monitoring and automatic recovery after interruption
- WebSocket feeds for live updates with REST reconciliation
- No missing data should silently be interpreted as zero

4.5 Historical Reconstruction

The initial research window is 90 days. The system should retain additional history where legally and technically available. Longer histories improve statistical confidence; market conditions change; traders may have different performance across regimes; short-term profitability may be accidental. The system must explicitly distinguish the 90-day selection window from the full historical evidence available for validation.

5. WALLET INTELLIGENCE ENGINE

5.1 Objective

Discover and analyse 10,000 candidate Polymarket wallets. Identify traders demonstrating persistent, measurable predictive advantages.

5.2 Wallet Discovery

Candidate wallets may be discovered through public trader leaderboards, market transaction histories, active-market trading activity, public wallet activity, counterparty and co-trading relationships, and previously identified high-performing wallets. Each wallet receives a permanent internal identifier (wallet_id, address, first_seen, last_seen, discovery_source, activity_status, historical_coverage, data_quality_score).

5.3 Wallet Reconstruction

Reconstruct trade history, position changes, entry/exit prices, realised/unrealised profits, capital utilisation, holding periods, market/event concentration and exposure changes. Transfers, redemptions, fees, partial fills and incomplete starting balances must be handled explicitly. A wallet's apparent profit must never be assumed to represent its complete economic return.

5.4 Performance Metrics

Metric	Purpose
Realised P&L	Actual closed-position profitability
Unrealised P&L	Current estimated position value
ROI	Return relative to a defined capital denominator
Win rate	Percentage of resolved positions profitable
Profit factor	Gross gains divided by gross losses
Maximum drawdown	Peak-to-trough equity decline
Average holding period	Trading duration
Turnover	Trading activity relative to capital
Concentration	Dependence on particular markets
Closing-line value	Entry quality relative to later prices
Calibration	Relationship between predicted probability and outcome
Consistency	Repeatability across independent events

5.5 Statistical Validation

A high win rate does not automatically establish skill. ORACLE must distinguish genuine predictive ability from randomness using Bayesian estimation (shrink extreme estimates with small samples), bootstrap analysis (event-level or market-cluster resampling), multiple-testing correction (false discovery rate procedures), out-of-sample validation, regime analysis across political, crypto, sports, economic and regulatory markets, and net profitability after fees, spread, slippage and execution costs.

5.6 Wallet Scoring Model

Component	Weight
Out-of-sample predictive skill	25%
Risk-adjusted net return	20%
Closing-line value	15%

Component	Weight
Consistency	15%
Timing advantage	10%
Liquidity and capacity	10%
Data quality	5%

These are configurable starting weights and must be validated. The score must never conceal small sample sizes, poor historical coverage, extreme concentration, negative out-of-sample results or unreliable reconstruction.

5.7–5.8 Wallet Intelligence Output & Behaviour Classification

Every wallet should have an analytical profile (ID, address, observation window, trades, volume, P&L, ROI, win rate, profit factor, drawdown, holding period, concentration, CLV, OOS performance, confidence, data completeness, strategy classification, intelligence score). Proposed classifications: Directional predictor, Market maker, Arbitrage trader, Event specialist, Momentum trader, Contrarian trader, Short/Long-duration trader, High-frequency participant, Unclassified. Behavioural similarity must not be presented as proof of common ownership.

6. NEWS AND EVENT INTELLIGENCE ENGINE

6.1 Objective

Continuously monitor external information and identify events that may affect prediction-market probabilities.

6.2 Information Sources

Official government announcements, election authorities, regulatory publications, company announcements, court decisions, sports governing bodies, established news publishers, financial news feeds, specialist industry publications, public social-media information. Source availability, licensing and access terms must be verified individually.

6.3–6.4 Information Processing & Event Extraction

Pipeline: Incoming Information → Source Verification → Duplicate Detection → Entity Extraction → Event Classification → Claim Extraction → Evidence Evaluation → Market Mapping → Probability Reassessment → Opportunity Evaluation. Each document analysed for entities, event type, location, participants, dates, quantities, claims, source identity, publication/observation time, confidence and contradictory evidence.

6.5 Source Reliability

Source Type	Treatment
Official primary source	High authority within its remit
First-party statement	Evidence of what was stated
Established journalism	Independent corroboration
Specialist analysis	Context and interpretation
Social-media claim	Unverified lead
Anonymous rumour	Low-confidence evidence
AI-generated summary	Not an independent source

6.6–6.7 Event Graph & Market Mapping

Create a graph of relationships: SOURCE → REPORTS → EVENT → INVOLVES → ENTITY → AFFECTS → MARKET → CONTAINS → OUTCOME. Additional relationships include SOURCE_CORROBORATES_SOURCE, CLAIM_CONTRADICTS_CLAIM, EVENT_PRECEDES_EVENT, WALLET_TRADED_MARKET, EVENT_CHANGES_PROBABILITY, MARKET_DEPENDS_ON_EVENT. Every relationship retains provenance, timestamp and confidence. The agent must understand exact market resolution conditions; a headline that appears relevant is not necessarily relevant to the market's actual contractual question. Resolution semantics are mandatory.

7. PROBABILITY INTELLIGENCE ENGINE

7.1 Objective

Estimate independent probabilities for prediction-market outcomes. The system must not blindly accept the displayed market price as the correct probability.

7.2 Forecasting Architecture

Model	Purpose
Historical base-rate model	Reference-class probabilities
Bayesian model	Evidence updating and uncertainty
Statistical prediction model	Structured feature-based forecasts
Domain-specific model	Specialist event forecasting
Information extraction model	Convert news into structured evidence
Market benchmark	Compare forecasts against market prices
Ensemble model	Combine validated predictions

7.3 Bayesian Probability Updating

P(H|E) = [P(E|H) × P(H)] / P(E), where H is the event hypothesis and E is new evidence. Evidence must be assessed for independence. Five publishers repeating the same original report do not constitute five independent confirmations.

7.4 Forecast Output

Every prediction must include market_id, outcome, probability, lower_bound, upper_bound, confidence, model_id, model_version, forecast_timestamp, evidence_references and invalidation_conditions. The confidence field must have a documented statistical interpretation and must not be confused with the event probability.

7.5 Forecast Evaluation

Brier Score: BS = (1/N) Σ (p■ – y■)² — Lower values are better.
Log Loss: LL = –(1/N) Σ [y■ log(p■) + (1–y■) log(1–p■)]
Calibration: Measure whether events forecast at 70% actually resolve YES approximately 70% of the time. Maintain calibration dashboards by model, category, forecast horizon, market liquidity and confidence band. The system must identify overconfidence and automatically reduce exposure when calibration deteriorates.

8. MISPRICING AND OPPORTUNITY DETECTION

8.1 Directional Mispricing

A market is potentially mispriced when the independent probability estimate differs materially from the executable price. For a YES contract: EV = P(YES) − C, where C is the executable acquisition price. This is gross expected value per share under a $1 payout assumption. Net expected value must deduct fees, spread, slippage, expected market impact and other execution costs.

8.2 Example

Independent probability 72%, executable YES price 58% → Gross expected edge 14 percentage points. This is a candidate opportunity, not automatically an executable trade. The agent must validate liquidity, uncertainty, settlement rules and costs.

8.3 Opportunity Classification

Classification	Description
Directional mispricing	Independent probability exceeds executable price
Reverse mispricing	Independent probability below market valuation
Complete-set arbitrage	All complementary outcomes purchasable below guaranteed payout
Related-market arbitrage	Structural pricing inconsistency between related contracts
Temporal opportunity	Market adjustment appears delayed
Liquidity opportunity	Pricing may be distorted by insufficient depth

Related-market discrepancies must not be described as guaranteed arbitrage unless payout and settlement equivalence are proven.

9. CROSS-OUTCOME ARBITRAGE ENGINE

9.1–9.2 Complete-Set Arbitrage

Detect price inconsistencies between mutually exclusive outcomes. Example: three mutually exclusive exhaustive outcomes with executable asks $0.32, $0.29, $0.31 (total $0.92). Gross profit = $0.08, ROI ≈ 8.70%. This is illustrative.

9.3 Arbitrage Verification

- Every possible outcome is covered and outcomes are mutually exclusive
- Outcome tokens belong to compatible market conditions; settlement rules are compatible
- No omitted cancellation, void, tie or other outcome exists
- Every leg has sufficient executable depth; fees and slippage included
- Minimum order sizes and tick sizes are respected
- Execution and settlement risks are assessed

9.4 Multi-Leg Execution

Identify Candidate → Validate Market Structure → Fetch Full Order Books → Calculate Net Return → Risk and Cost Checks (Fail → Reject) → Calculate Leg Quantities → Execute Controlled Sequence → Monitor Partial Fills → Reconcile All Legs → Confirm Residual Exposure. Multi-leg execution must account for legging risk. A partially filled arbitrage is not necessarily an arbitrage anymore.

10. KELLY CAPITAL ALLOCATION ENGINE

10.1–10.3 Binary Kelly Formula & Example

For a binary contract: f* = (b·p − q) / b, where f* is optimal fraction of bankroll, b is net payout odds, p is probability of success, q = 1−p. For a YES contract bought at price c: b = (1−c)/c.

Example: Bankroll $10,000, entry price $0.58, estimated probability 0.72 → b ≈ 0.7241, q = 0.28, f* ≈ 0.3336 (33.36% full Kelly). Quarter Kelly: f_actual ≈ 8.34% → position ≈ $834. This is an unconstrained illustration before fees, slippage, correlation and probability uncertainty.

10.4 Fractional Kelly

Initial recommended configuration: KELLY_MULTIPLIER = 0.25. The multiplier is configurable and must be validated empirically.

10.5 Portfolio Kelly

Independent Kelly calculations can create dangerous aggregate exposure when several positions depend on the same event. The production system must account for correlated outcomes, related political events, common economic drivers, cryptocurrency market dependence, shared resolution authorities, portfolio volatility, liquidity and capital lock-up. Use a constrained portfolio optimiser rather than treating every position as independent.

10.6 Risk Limits

All values configurable: MAX_SINGLE_MARKET_EXPOSURE, MAX_SINGLE_EVENT_EXPOSURE, MAX_CATEGORY_EXPOSURE, MAX_DAILY_LOSS, MAX_PORTFOLIO_DRAWDOWN, MAX_ORDER_NOTIONAL, MAX_OPEN_ORDER_COUNT, MAX_MARKET_PARTICIPATION, MAX_KELLY_FRACTION, MAX_MODEL_UNCERTAINTY.

10.7 No-Trade Conditions

- Expected net value is insufficient
- The model is outside its validated domain
- Market resolution criteria are ambiguous
- Liquidity is inadequate or quotes are stale
- Portfolio exposure is excessive
- Required evidence is unreliable
- Risk controls are unavailable or trading eligibility is uncertain

11. AUTONOMOUS AGENT ARCHITECTURE

11.1–11.2 Objective & Capabilities

Build a persistent autonomous agent capable of continuously observing, investigating, evaluating and managing prediction-market opportunities. The agent must maintain state and structured memory. It must not be a simple sequence of fixed if-then rules. Capabilities include discovering opportunities, deciding which events require investigation, selecting sources, retrieving wallet intelligence, identifying affected markets, challenging contradictory information, requesting probability estimates, comparing against executable prices, calculating risk-adjusted allocations, monitoring positions, detecting invalidated theses, recommending exits, and learning from resolved outcomes.

11.3 Agent Components

Component	Responsibility
Planner	Research task selection
Evidence manager	Evidence and source tracking
Event analyser	Event interpretation
Market mapper	Market resolution alignment
Wallet analyst	Historical trader intelligence
Forecast coordinator	Probability model orchestration
Opportunity evaluator	EV and arbitrage evaluation
Risk client	Independent risk approval
Execution monitor	Order and position monitoring
Learning service	Outcome evaluation

11.4 Agent Decision Lifecycle

Observe → Detect Relevant Change → Investigate Sources → Identify Affected Markets → Analyse Wallet Behaviour → Estimate Independent Probability → Challenge Assumptions → Calculate Expected Value → Evaluate Arbitrage → Calculate Kelly Allocation → Independent Risk Approval → Decision (Reject / Watch / Approve) → Submit Order Intent → Monitor and Reconcile → Evaluate Outcome → Update Intelligence.

11.5 Persistent Memory

Memory must be structured and stored externally. Categories: wallet performance history, strategy classifications, event dossiers, market resolution histories, source reliability, historical probability estimates, trading decisions, failed predictions, strategy performance, operator instructions, model evaluation history. Each item includes timestamp, source, confidence and update policy.

11.6 Agent Reasoning Requirements

The agent must answer: What happened? What is known? What remains uncertain? Which markets are affected? What is the market currently pricing? What is the independently estimated probability? What evidence supports or contradicts the estimate? What would invalidate the thesis? Is the opportunity executable? What is the risk-adjusted position size? Should the system trade, wait or reject?

11.7 AI Safety Boundaries

The LLM must never: access private signing keys; modify hard risk limits; directly submit orders; override market eligibility; treat external web content as system instructions; invent missing evidence; convert an unverified claim into a confirmed fact. All financial actions require independent deterministic validation.

12. EXECUTION ENGINE

12.1 Execution Modes

Mode	Function
Research	Data and analytics only
Replay	Historical simulation
Paper	Simulated trading
Shadow	Live opportunity detection without orders
Guarded live	Approved real-money execution

Live mode is disabled by default.

12.2–12.3 Execution Architecture & Order Lifecycle

Autonomous Agent → Order Intent → Policy Validation → Risk Approval → Execution Gateway → Polymarket CLOB → Order Reconciliation. Order states: CREATED → POLICY_PENDING → APPROVED → SUBMITTED → ACKNOWLEDGED → (PARTIALLY_FILLED | FILLED | CANCEL_PENDING | CANCELLED | REJECTED | EXPIRED).

12.4 Execution Requirements

- Limit orders by default; marketable limit orders only within configured collars
- Order-book depth validation and slippage protection
- Idempotency controls and duplicate-order prevention
- Partial-fill management and automatic reconciliation
- Stale-order cancellation, emergency stop, execution logging

12.5 Credential Security

Private keys must be isolated from LLM context, frontend code, general application logs, notebooks, source repositories and unrestricted application workers. Use a dedicated signing boundary, scoped credentials and a separately controlled execution service.

13. DATABASE ARCHITECTURE

13.1 Database Strategy

PostgreSQL is the primary system of record. TimescaleDB or partitioned PostgreSQL handles time-series records. Parquet and DuckDB support historical analytical workloads. Object storage retains raw source payloads. Valkey/Redis supports caching and queues but is not the authoritative financial ledger.

13.2 Core Tables

Primary tables: wallets, wallet_trades, markets, forecasts, opportunities, orders. Additional required tables: wallet_positions, wallet_scores, orderbook_snapshots, source_documents, claims, events, market_event_links, risk_decisions, fills, settlements, audit_events, model_versions, model_evaluations, portfolio_snapshots, agent_memory.

13.3 Audit Architecture

All important decisions must be recorded in an append-only audit system. Every financial decision must be traceable to: trigger, source information, market snapshot, wallet intelligence, probability estimate, expected value, risk calculation, approval, order, fill and settlement. Use hashes and immutable storage to detect alteration.

14. API AND SERVICE INTERFACES

14.1 Proposed Internal APIs

Endpoint	Function
GET /markets	Retrieve markets
GET /markets/{id}	Retrieve market details
GET /wallets	Retrieve wallet registry
POST /wallets/discover	Start wallet discovery
GET /wallets/{id}/profile	Retrieve wallet profile
POST /wallets/analyse	Analyse wallet performance
POST /events/ingest	Ingest event information
GET /events/{id}/evidence	Retrieve evidence
POST /forecasts	Generate forecast
POST /forecasts/score	Evaluate forecast
POST /opportunities/evaluate	Evaluate opportunity
POST /risk/evaluate	Request risk approval
POST /order-intents	Submit approved order intent
GET /orders/{id}	Retrieve order state
POST /risk/kill-switch	Activate emergency controls
GET /audit	Retrieve decision records

14.2–14.3 Order Intent Contract & Error Handling

Order intent includes intent_id, opportunity_id, mode, market_id, token_id, side, limit_price, quantity, max_slippage_bps, forecast_id, risk_decision_id, idempotency_key. The execution service independently validates the entire request. Stable error codes: INVALID_MARKET_RULES, STALE_QUOTE, INSUFFICIENT_DEPTH, MODEL_OUT_OF_DOMAIN, RISK_LIMIT_EXCEEDED, ELIGIBILITY_BLOCKED, DUPLICATE_INTENT, VENUE_UNAVAILABLE, RECONCILIATION_REQUIRED, MANUAL_REVIEW_REQUIRED.

15. SECURITY AND GOVERNANCE

15.1 Security Principles

- Least privilege, credential isolation, network segmentation
- Encryption in transit and at rest, secret rotation
- Role-based access control, immutable audit trails
- Independent execution controls, regular security testing

15.2 User Roles

Role	Permissions
Viewer	Read-only dashboard
Research analyst	Research and analysis
Quant administrator	Model configuration
Risk operator	Risk limits and controls
Execution operator	Authorised execution operations
System administrator	Infrastructure management

Sensitive operations should require dual approval.

15.3 Regulatory Considerations

Before enabling live trading, conduct a formal assessment of South African regulatory requirements, Polymarket terms and geographical eligibility, cross-border financial activity, tax treatment, applicable gambling/derivatives/financial-services classifications, data protection and privacy, and sanctions/restricted-party obligations. Technical access does not establish legal permission. ORACLE must not circumvent platform restrictions or jurisdictional controls.

16. MONITORING AND OBSERVABILITY

16.1 Required Operational Metrics

Data Ingestion: API latency, request failures, rate-limit events, missing data, duplicates, WebSocket disconnections.
Wallet Intelligence: Wallets discovered/analysed, historical completeness, reconstruction exceptions, statistical confidence, score distribution.
Probability Engine: Brier score, log loss, calibration, forecast drift, model disagreement, out-of-domain predictions.
Opportunity Engine: Opportunities detected, net EV, available depth, quote age, false-positive rate, rejected opportunities.
Risk Engine: Portfolio exposure, Kelly allocation, drawdown, daily losses, concentration, blocked orders.
Execution Engine: Order latency, fill rate, slippage, rejected orders, cancellation rate, reconciliation discrepancies.

16.2 Emergency Response

SEV-1 (Unauthorised order, credential compromise, runaway execution, material account discrepancy): Immediately disable new risk, preserve evidence and initiate credential/account incident procedures.
SEV-2 (Reconciliation failure, stale market data with execution enabled, risk service unavailable): Block new orders and reconcile.
SEV-3 (Source outage, model drift, ingestion degradation): Switch to degraded or read-only operation.

17. TESTING AND VALIDATION

17.1–17.2 Unit & Integration Testing

Unit: P&L reconstruction, probability/Kelly/fee/slippage calculations, risk limits, order states, portfolio exposure. Integration: Market API connectors, wallet/news ingestion, database operations, forecast generation, opportunity calculations, risk approval, execution reconciliation.

17.3–17.4 Historical Replay & Walk-Forward Testing

Reconstruct historical market conditions using point-in-time information. Prevent look-ahead bias. A historical simulation must not use information that was unavailable when the hypothetical decision would have occurred. Use chronological training, validation and testing periods. Avoid placing related events in both training and test sets where this creates leakage.

17.5–17.6 Paper & Shadow Testing

Paper: Run continuously without real capital. Measure executable opportunity frequency, simulated fill quality, expected vs realised returns, model calibration, execution latency, drawdown, capital utilisation. Simulated fills must be conservative. Shadow: Generate real-time order intentions without submitting them. Compare predicted vs actual executable price, expected vs actual subsequent market movement, net hypothetical return.

17.7 Go-Live Criteria

- Data completeness is measured and acceptable
- Wallet ranking is statistically validated
- Probability calibration meets predefined standards
- Backtests pass independent review
- Paper execution is reliable; risk controls pass automated tests
- Order reconciliation is reliable; security review is complete
- Legal eligibility is confirmed; an authorised operator enables live mode

18. DEPLOYMENT ARCHITECTURE

18.1 Environment Structure

DEVELOPMENT → RESEARCH → STAGING → PRODUCTION RESEARCH → PAPER / SHADOW → GUARDED LIVE EXECUTION. Each environment must have separate credentials and configuration.

18.2 Infrastructure

Recommended initial deployment: Linux server, Docker containers, PostgreSQL, Valkey, Object storage, FastAPI backend, Next.js dashboard, Monitoring stack, Encrypted backup storage. Kubernetes should only be introduced when actual scale and operational requirements justify it.

18.3 Recovery

Implement database backups, point-in-time recovery, object-store versioning, infrastructure-as-code, service health checks, automatic restart, execution reconciliation after outages. Never assume local order state is correct after a service interruption. Reconcile against venue records before enabling additional exposure.

19. DEVELOPMENT ROADMAP

Phase 0 — Project Foundation

Repository, development environment, database schema, API framework, deployment configuration, security baseline.

Phase 1 — Wallet Intelligence

Wallet discovery, 10,000-wallet registry, historical data ingestion, trade reconstruction, performance analytics, statistical validation, wallet leaderboard.

Phase 2 — Event Intelligence

News ingestion, source verification, entity extraction, event classification, event graph, market mapping.

Phase 3 — Probability Intelligence

Base-rate models, Bayesian updating, forecast ensemble, calibration dashboard, forecast evaluation.

Phase 4 — Opportunity Intelligence

Mispricing scanner, order-book analysis, complete-set arbitrage, related-market analysis, fee and slippage models.

Phase 5 — Risk Management

Fractional Kelly, portfolio Kelly, correlation controls, capital limits, drawdown controls, risk approval service.

Phase 6 — Autonomous Agent

Persistent agent, event monitoring, research planner, evidence manager, decision engine, agent memory, decision ledger.

Phase 7 — Paper Execution

Paper order gateway, simulated fills, position management, performance tracking, reconciliation.

Phase 8 — Guarded Live Execution

Isolated execution gateway, credential management, operator approvals, emergency controls, live monitoring, independent security and compliance approval.

19.1 Indicative Development Schedule

Workstream	Indicative Duration
Foundation	1–3 weeks
Wallet intelligence	3–6 weeks
Event intelligence	5–8 weeks
Probability intelligence	7–10 weeks
Opportunity and risk	8–13 weeks
Agent and paper execution	11–16 weeks
Shadow and hardening	16–20+ weeks

These are overlapping planning estimates for an experienced engineering team, not guaranteed delivery dates. Prospective validation and regulatory review may extend the schedule.

20. REPOSITORY STRUCTURE

Recommended standalone GitHub repository structure:

```
oracle/
■■■ apps/          (dashboard/, api/)
■■■ services/      (market_connectors/, wallet_intelligence/, news_ingestion/,
■                   event_intelligence/, forecasting/, opportunity_engine/,
■                   risk_engine/, agent_orchestrator/, execution_gateway/,
■                   reconciliation/)
■■■ packages/      (schemas/, venue_sdk/, quant_core/, audit/)
■■■ data/          (fixtures/, migrations/)
■■■ research/      (notebooks/, evaluations/, reports/)
■■■ infra/         (docker/, terraform/, monitoring/)
■■■ tests/         (unit/, contract/, integration/, replay/, security/)
■■■ docs/          (architecture/, runbooks/, compliance/, model_cards/)
■■■ .github/       (workflows/)
■■■ pyproject.toml
■■■ docker-compose.yml
■■■ README.md
■■■ LICENSE
```

21. ACCEPTANCE CRITERIA

ORACLE is considered ready for its research release when:

- The wallet discovery system can maintain a 10,000-address candidate registry
- Historical data coverage is measurable; wallet activity can be reconstructed with documented limitations
- Statistical rankings include uncertainty and out-of-sample evidence
- News information can be associated with relevant market questions
- Probability forecasts are versioned and measurable
- Opportunities are evaluated against executable prices
- Arbitrage calculations account for complete outcome coverage and costs
- Fractional Kelly calculations are independently tested; portfolio exposure is constrained
- The autonomous agent maintains persistent state; every decision is auditable
- Paper trading can operate continuously; no live order can bypass risk approval

Live execution readiness requires additional prospective, security, legal and operational acceptance.

22. NON-NEGOTIABLE ENGINEERING INVARIANTS

1. The agent must never directly access private signing credentials.
2. No LLM output can independently authorise a financial transaction.
3. No trade can execute without fresh market data.
4. No arbitrage can be classified as guaranteed without settlement verification.
5. No wallet can be classified as statistically skilled based solely on headline profits.
6. No probability estimate can be accepted without recording its model and inputs.
7. No risk limit can be modified by the autonomous agent.
8. No position can bypass portfolio-level exposure controls.
9. No unresolved reconciliation discrepancy can silently permit additional exposure.
10. Every financial action must have a complete decision trace.
11. Every model must be evaluated prospectively.
12. The system must be capable of refusing to trade.

23. FUTURE DEVELOPMENT

Once the core architecture is validated, ORACLE can be extended with:

23.1 Cross-platform Intelligence

Monitor additional prediction-market venues where access, data quality and legal eligibility permit.

23.2 Advanced Trader Clustering

Identify behavioural relationships between wallets using graph analysis and temporal transaction patterns.

23.3 Information Propagation Analysis

Estimate how quickly information spreads between publishers, social channels, wallets and markets.

23.4 Predictive Market Impact

Study how different classes of information affect market repricing and liquidity.

23.5 Strategy Discovery

Use systematic research to identify new strategies rather than relying on predefined trading patterns.

23.6 Autonomous Research

Allow ORACLE to formulate research questions, conduct bounded investigations, test hypotheses and submit proposed model improvements for validation.

23.7 Portfolio-wide Optimisation

Develop more advanced portfolio allocation across correlated events and multiple prediction-market venues.

24. FINAL ENGINEERING DIRECTIVE

ORACLE must be engineered as an autonomous intelligence platform with a quantitatively verifiable financial decision system. Its advantage must come from the combination of historical trader intelligence, independent probability estimation, continuous information monitoring, market microstructure analysis, statistical edge validation, arbitrage detection, portfolio optimisation, autonomous reasoning and disciplined execution.

The system must distinguish information from evidence, correlation from causation, apparent profitability from repeatable edge, and theoretical arbitrage from executable arbitrage. It must continuously measure its own performance and remain capable of declining opportunities when the evidence is insufficient.

The objective is not to trade more frequently. It is to make better decisions, identify genuine information advantages and allocate capital only when the measurable expected return justifies the risk.

END OF ORACLE ENGINEERING BLUEPRINT v1.0

Document status: Proposed engineering specification.
This document defines the target architecture and implementation requirements. It does not represent a completed software deployment, verified trading performance, guaranteed arbitrage or legal authorisation.